```
use OnlineRetail;

--Check that all rows were added from known number of rows

select
count(*)
from onret1;

select
count(*)
from onret2;

select
count(distinct(stockcode))
from onret1;
```

```
select
count(stockcode)
from onret1
group by stockcode;

select top 100 * from onret1;
select top 100 * from onret2;

select * from onret1 where invoice like 'C%';
select * from onret2 where invoice like 'C%';

-- Count how many cancelled orders in each table

select count(*) from onret1 where invoice like 'C%';
select count(*) from onret2 where invoice like 'C%';
```

```
-- delete cancelled orders

delete from onret1 where invoice like 'C%';
delete from onret2 where invoice like 'C%';

select count(distinct(customer_ID)) from onret1;

select count(distinct(customer_ID)) from onret2;

select count(*) from onret1 where customer_id is null;
select count(*) from onret2 where customer_id is null;
```

-- delete records where price=0 but quantity is not negative as these are likely to be erroneous data entries that do not offer any value to insights

--Also include customer_id = 'guest' (formerly null) further indicating erroneous data entry.

```
delete from onret where price = 0 and Quantity > 0 and customer_id = 'guest';

delete from onret where price = 0 and Quantity > 0;

select
*
from onret
where price = 0
order by customer_ID;
```

```
--merge both tables into one
drop table onret;

select *
into onRet
from onret1
union
select * from onRet2;



select
count(*)
from onret
where customer_ID is null;
```

```
-- There are 234437 instances of null customer_ID so I will replace these with 'Guest'. Too many rows to delete and ignore

alter table onret
alter column customer_ID varchar(50);

UPDATE onret
SET customer_ID = 'guest'
WHERE customer_ID is null;

select * from onret;

select top 10 * from onret order by quantity desc;
```

```
alter table onret1
alter column quantity int;
alter table onret2
alter column quantity int;


select * from onret1 where quantity = '80995'
select * from onret2 where quantity = '80995'

SELECT top 100 * from onret;

--Count how many orders have been duplicated
```

```
select count(*) from
(
        select
      invoice,
      StockCode
   from onRet
   group by invoice, stockcode
   having count(*)>1
)t
--there are 11045 records duplicated a varying number of times.
```

```
select
count(*) cnt,
   invoice,
   stockcode
from onRet
group by invoice, stockcode
having count(*)>1
order by count(*) desc;
```

```
-- This shows how many times each duplicate has been duplicated

select sum(cnt) from
(
   select
   count(*) cnt,
      invoice,
      stockcode
   from onRet
   group by invoice, stockcode
   having count(*)>1
)t
```

-- This is total number of duplicates = 22466

-- 22466 - 11045 = 11421 rows need to be deleted

-- This windowed sub-query finds all the duplicated rows but keeps one record in place. I will use this to delete all but one duplicated order

```
select * from
(
   select
      invoice,
      stockcode,
      row_number() over(partition by invoice, stockcode order by country) cnt
   from onret
)t
where cnt>1;
```

```
-- Backup the main table for now

select * into onRetBackup from onRet;

-- Execute the deletion using a cte on a windowed query using row_number() grouped by the natural key invoiceID and stockcode
-- and only deleting row numbers > 1 which means you keep one of each duplicated order

with duplicates as
(
        select
       *,
       row_number() over(partition by invoice, stockcode order by invoice) rn
   from onRet
)
delete from duplicates where rn>1;
```

-- Deduplication complete

-- I will now create a customer table to hold information for each customer

```
select
customer_id,
sum(totalrevenue) totalRevenue,
sum(quantity) itemsOrdered,
min(invoiceDate) firstOrderDate,
max(invoicedate) lastOrderDate,
count(distinct invoice) numberOfOrders,
sum(totalrevenue) / cast(count(distinct invoice)as float) avgOrderCost
into customers2
from onret
group by Customer_ID;
```

100 % No issues found

Results Messages

	customer_id	totalRevenue	itemsOrdered	firstOrderDate	lastOrderDate	numberOfOrders	avgOrderCost
1	14646	526679	367037	2009-12-02 16:52:00.000	2011-12-08 12:12:00.000	151	3487.94039735099
2	15446	321	247	2009-12-13 13:53:00.000	2010-11-28 11:59:00.000	3	107
3	15306	3416	3074	2010-02-18 17:12:00.000	2011-10-06 12:56:00.000	9	379.555555555556
4	17422	3514	3538	2010-06-23 10:19:00.000	2011-11-23 13:27:00.000	11	319.454545454545
5	13648	1716	960	2009-12-01 18:13:00.000	2010-11-04 14:46:00.000	8	214.5
6	15132	953	679	2011-03-22 12:02:00.000	2011-11-21 17:10:00.000	5	190.6
7	14529	441	296	2011-08-21 14:10:00.000	2011-10-10 15:27:00.000	3	147
8	12886	1341	770	2011-05-04 11:21:00.000	2011-10-03 10:09:00.000	2	670.5
9	14980	136	79	2009-12-02 14:09:00.000	2009-12-02 14:09:00.000	1	136
10	17414	2242	6256	2009-12-03 16:49:00.000	2011-10-27 12:13:00.000	6	373.666666666667
11	15462	724	495	2010-09-20 10:23:00.000	2011-09-06 13:11:00.000	4	181
12	14462	2062	1028	2010-10-03 14:13:00.000	2011-10-04 15:55:00.000	12	171.833333333333
13	16584	3117	1913	2010-02-16 10:41:00.000	2011-09-20 13:45:00.000	5	623.4
14	13291	3016	1573	2010-02-11 18:36:00.000	2011-07-07 10:36:00.000	11	274.181818181818
15	16023	665	358	2011-03-18 13:32:00.000	2011-10-24 16:34:00.000	3	221.666666666667

Query executed successfully.

--CODE FROM HERE SHOWS SOME OF THE PROCESSES I WENT THROUGH TO GET TO THE DELETION QUERIES ABOVE. TO...

```
select * from
(
```

```
    select
        *,
        row_number() over(partition by invoice, stockcode order by invoice) rn
    from onRet
)t
order by invoice, stockcode, rn desc;
```

```
select top 1000 count(invoice) cnt, invoice, stockcode from onret group by invoice, stockcode order by 1
desc;

select sum(cnt) from
(
```

```
select count(invoice) cnt, stockcode from onret group by invoice, stockcode
)t



select
   count(invoice),
   stockcode
from onret
group by invoice, stockcode
order by 1 desc;



-- identify if there are any repeated orders present for deduplication
```

```
select
   count(invoice) cnt,
   invoice,
   stockcode
from onret
group by invoice, stockcode
having count(invoice)>1
order by cnt desc;
```

--Count how many duplicated records of sales there are

```
select sum(cnt) from
```

```
(
   select
      count(invoice) cnt,
      stockcode
   from onret
   group by invoice, stockcode
)t
 where cnt>1;
```

-- There are 22466 duplicated sales

-- This windowed sub-query finds all the duplicated rows but keeps one record in place. I will use this to delete all but one duplicated order

```
select * from
(
   select
      row_number() over(partition by invoice, stockcode order by country) cnt
   from onret
)t
where cnt>1;


-- Backup the database in this state using a CTAS

select *
into onRetBACKUP
from onret;
```

```
drop table onretC;



select
*
into onRetC
from
(
   select
      row_number() over(partition by invoice, stockcode order by country) cnt,
      *
   from onret
)t
where cnt>1;
```

```
select sum(cnt) from
(
   select
      count(invoice) cnt,
      invoice,
      stockcode
   from onretC
   group by invoice, stockcode
   order by cnt desc
)t
where cnt>1 ;
```

```
select
    invoice,
    stockcode
from onretC
group by invoice,stockcode
having stockcode = 21791
order by invoice, stockcode;



select top 100 * from onRetC;


--...TO HERE


--Now I do some transformations to make proceeding caculations easier and for when I connect tables to Tableau
```

```
 -- add revenue per invoice column

alter table onret
add totalRevenue int;



UPDATE onret
SET totalrevenue = quantity*price;


select top 100 * from onret;
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer_ID	Country	totalRevenue
1	489780	21930	JUMBO STORAGE BAG SKULLS	10	2009-12-02 11:43:00.000	1.95000004768372	13654	United Kingdom	19
2	489780	22083	PAPER CHAIN KIT RETRO SPOT	12	2009-12-02 11:43:00.000	2.95000004768372	13654	United Kingdom	35
3	489780	22086	PAPER CHAIN KIT 50'S CHRISTMAS	12	2009-12-02 11:43:00.000	2.95000004768372	13654	United Kingdom	35
4	489782	17109A	FLOWER FAIRY INCENSE LILY	4	2009-12-02 11:45:00.000	1.69000005722046	17189	United Kingdom	6
5	489782	17109C	FLOWER FAIRY INCENSE ROSE	4	2009-12-02 11:45:00.000	1.69000005722046	17189	United Kingdom	6
6	489782	20724	RED SPOTTY CHARLOTTE BAG	6	2009-12-02 11:45:00.000	0.850000023841858	17189	United Kingdom	5
7	489782	20728	LUNCH BAG CARS BLUE	2	2009-12-02 11:45:00.000	1.64999997615814	17189	United Kingdom	3
8	489782	20751	FUNKY WASHING UP GLOVES ASSORTED	1	2009-12-02 11:45:00.000	2.09999990463257	17189	United Kingdom	2
9	489782	20752	RETRO BLUE SPOTTY WASHING UP GLOVES	3	2009-12-02 11:45:00.000	2.09999990463257	17189	United Kingdom	6
10	489782	20753	RETRO PINK SPOT WASHING UP GLOVES	2	2009-12-02 11:45:00.000	2.09999990463257	17189	United Kingdom	4
11	489782	20754	RETRO RED SPOTTY WASHING UP GLOVES	1	2009-12-02 11:45:00.000	2.09999990463257	17189	United Kingdom	2
12	489782	20940	SET/6 GARDEN GNOME T-LIGHT CANDLES	6	2009-12-02 11:45:00.000	2.95000004768372	17189	United Kingdom	17
13	489782	20941	FROG CANDLE	12	2009-12-02 11:45:00.000	1.25	17189	United Kingdom	15
14	489782	20998	ROSE DU SUD OVEN GLOVE	4	2009-12-02 11:45:00.000	2.95000004768372	17189	United Kingdom	11
15	489782	21556	CERAMIC STRAWBERRY MONEY BOX	3	2009-12-02 11:45:00.000	2.54999995231628	17189	United Kingdom	7

Query executed successfully.

```
select
   *,
   sum(quantity*price) totalRevenue
into RevenuePerInvoice
from onret;
```

```
select top 20 invoice, sum(quantity*price) totalRevenue from onret group by invoice;

select count(distinct(Customer_ID)) from onret;

--create a new table to hold information about each unique customer using CTAS

select
   customer_ID customer_ID,
   sum(totalrevenue) totalRevenue,
   count(distinct invoice) totalOrders,
   max(invoiceDate) lastOrder,
   sum(quantity) totalItemsOrdered,
   sum(totalrevenue)/sum(quantity) averageOrderCost
into customers
```

```
from onret
group by Customer_ID;



--drop table customers;



WITH cohorts AS (
  SELECT
    Customer_ID,
    cast(MIN(DATEPART(year, InvoiceDate))as varchar) +' '+ cast(MIN(Datepart(month, invoicedate))as
varchar) AS CohortMonth
  FROM onRet
```

```
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  cast(datepart(year,invoiceDate)as varchar)+' '+cast(DATEPART(month, InvoiceDate)as varchar) AS
PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
FROM onRet a
JOIN cohorts b on a.Customer_ID = b.Customer_ID
GROUP BY CohortMonth, cast(datepart(year,invoiceDate)as varchar)+' '+cast(DATEPART(month,
InvoiceDate)as varchar)
order by CohortMonth, PurchaseMonth;
```

Results | Messages

	CohortMonth	PurchaseMonth	ActiveCustomers
1	2009 1	2009 12	445
2	2009 1	2010 1	338
3	2009 1	2010 10	251
4	2009 1	2010 11	276
5	2009 1	2010 12	238
6	2009 1	2010 2	236
7	2009 1	2010 3	272
8	2009 1	2010 4	243
9	2009 1	2010 5	222
10	2009 1	2010 6	243
11	2009 1	2010 7	235
12	2009 1	2010 8	233
13	2009 1	2010 9	234
14	2009 1	2011 1	271
15	2009 1	2011 10	186

Query executed successfully. | DESKTOP

```
WITH cohortsYEARS AS (
```

```
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onRet
  where datepart(year,InvoiceDate)=2010
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
FROM onRet a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
where datepart(year,invoicedate)=2010
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;
```

Results | Messages

	CohortMonth	PurchaseMonth	ActiveCustomers
1	1	1	721
2	1	2	263
3	1	3	341
4	1	4	316
5	1	5	285
6	1	6	307
7	1	7	285
8	1	8	270
9	1	9	285
10	1	10	317
11	1	11	335
12	1	12	253
13	2	2	510
14	2	3	144
15	2	4	141

Query executed successfully. | DESKT

-- previous CTE tables create cohort retention data however because date_trunc does not work on this version of SQL, months from different years are treated as the same time point

-- I will therefore create three new tables, one for each year and run the cohort retention analysis on them seperately.

```
--drop table onret2009

--drop table onret2010

--drop table onret2011


select *

into onret2009

from onRet

where datepart(year, invoicedate) = 2009;


select * from onret2009;
```

```
select *
into onret2010
from onRet
where datepart(year, invoicedate) = 2010;

select *
into onret2011
from onRet
where datepart(year, invoicedate) = 2011;

--Here I do a cohort analysis for each year seperatley

WITH cohortsYEARS AS (
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
```

```
  FROM onRet2009
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
FROM onRet2009 a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;
```

```
WITH cohortsYEARS AS (
  SELECT
```

```
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onret2010
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
FROM onret2010 a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;
```

```
WITH cohortsYEARS AS (
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onret2011
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
FROM onret2011 a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;
```

```
--Or use the original CTEs from the one onRet table and use a where clause to seperate the years

WITH cohortsYEARS AS (
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onRet
  where datepart(year,InvoiceDate)=2009
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
  into cohortRetention2009
FROM onRet a
```

```
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
where datepart(year,invoicedate)=2009
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;
```

```
WITH cohortsYEARS AS (
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onRet
  where datepart(year,InvoiceDate)=2010
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
  into cohortRetention2010
FROM onRet a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
where datepart(year,invoicedate)=2010
```

```
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;



WITH cohortsYEARS AS (
  SELECT
    Customer_ID,
    MIN(Datepart(month, invoicedate)) AS CohortMonth
  FROM onRet
  where datepart(year,InvoiceDate)=2011
  GROUP BY Customer_ID
)
SELECT
  CohortMonth,
  DATEPART(month, InvoiceDate) AS PurchaseMonth,
```

```
  COUNT(DISTINCT a.Customer_ID) AS ActiveCustomers
  into cohortRetention2011
FROM onRet a
JOIN cohortsYEARS b on a.Customer_ID = b.Customer_ID
where datepart(year,invoicedate)=2011
GROUP BY CohortMonth, DATEPART(month, InvoiceDate)
order by CohortMonth, PurchaseMonth;

--I now have 3 seperate cohort retention tables for each year ready for analysis and insights.

select * from cohortRetention2009 order by CohortMonth, purchasemonth;
select * from cohortRetention2010 order by CohortMonth, purchasemonth;
select * from cohortRetention2011 order by CohortMonth, purchasemonth;
```

```
--drop table cohortRetention2009

--drop table cohortRetention2010

--drop table cohortRetention2011


--Here are 2 quick window function queries that create two tables for 2010 and 2011 on how much
-- of a cohort are still active in numbers and as a percentage of the original cohort size




with retanalysis as

(

   select

   *,

   first_value(activeCustomers) over(partition by cohortmonth order by purchasemonth)
OriginalCohortSize

   from cohortRetention2010

)

select
```

```
    *,
    (cast(ActiveCustomers as float)/cast(OriginalCohortSize as float))*100 percentRetained
into CRA2010
from retanalysis;
```

Results | Messages

	CohortMonth	PurchaseMonth	ActiveCustomers	OriginalCohortSize	percentRetained
1	1	1	721	721	100
2	1	2	263	721	36.4771151178918
3	1	3	341	721	47.2954230235784
4	1	4	316	721	43.8280166435506
5	1	5	285	721	39.5284327323162
6	1	6	307	721	42.5797503467406
7	1	7	285	721	39.5284327323162
8	1	8	270	721	37.4479889042996
9	1	9	285	721	39.5284327323162
10	1	10	317	721	43.9667128987517
11	1	11	335	721	46.4632454923717
12	1	12	253	721	35.0901525658807
13	2	2	510	510	100
14	2	3	144	510	28.2352941176471
15	2	4	141	510	27.6470588235294

with retanalysis as

```
(
   select
   *,
   first_value(activeCustomers) over(partition by cohortmonth order by purchasemonth)
OriginalCohortSize
   from cohortRetention2011
)
select
   *,
   (cast(ActiveCustomers as float)/cast(OriginalCohortSize as float))*100 percentRetained
into CRA2011
from retanalysis;
```

100 % No issues found

Results Messages

	CohortMonth	PurchaseMonth	ActiveCustomers	OriginalCohortSize	percentRetained
1	1	1	742	742	100
2	1	2	263	742	35.44474393531
3	1	3	320	742	43.1266846361186
4	1	4	285	742	38.4097035040431
5	1	5	340	742	45.822102425876
6	1	6	312	742	42.0485175202156
7	1	7	291	742	39.2183288409704
8	1	8	286	742	38.544474393531
9	1	9	326	742	43.9353099730458
10	1	10	310	742	41.7789757412399
11	1	11	382	742	51.4824797843666
12	1	12	191	742	25.7412398921833
13	2	2	496	496	100
14	2	3	116	496	23.3870967741935
15	2	4	123	496	24.7983870967742

Query executed successfully. DESKTOP-1MKC00T\SQLEXPRESS ... DESKTOP

```
select * from CRA2011;
select * from CRA2010;
```

```
-- TRANSFORMATIONS END HERE AND NOW I DO SOME BUSINESS ANALYSIS OF THE DATA

--Now, here are some business analysis of the data

-- Top 10 customers by lifetime revenue and top 10 revenue

select top 10
   customer_ID,
   sum(totalrevenue) lifetimeRevenue
from onret
where Customer_ID <> 'guest'
group by Customer_ID
order by lifetimeRevenue desc;
```

Results | Messages

	customer_ID	lifetimeRevenue
1	18102	580478
2	14646	526679
3	14156	311410
4	14911	286492
5	17450	244581
6	13694	194982
7	17511	171254
8	16446	168471
9	16684	146806
10	12415	144082

```
-- Revenue trend -3-month rolling average.


select top 50 * from onret;


select
   datepart(year,invoicedate),
```

```
    datepart(month,invoicedate),
    sum(totalrevenue)
from onret
group by datepart(year,invoicedate), datepart(month,invoicedate)
order by datepart(year,invoicedate), datepart(month,invoicedate);
```

Results | Messages

	(No column name)	(No column name)	(No column name)
1	2009	12	794499
2	2010	1	633114
3	2010	2	534780
4	2010	3	807463
5	2010	4	659404
6	2010	5	634923
7	2010	6	724453
8	2010	7	629580
9	2010	8	675107
10	2010	9	895282
11	2010	10	1125598
12	2010	11	1410844
13	2010	12	790524
14	2011	1	669006
15	2011	2	506578
16	2011	3	695147
17	2011	4	516726
18	2011	5	748654
19	2011	6	735137
20	2011	7	696588
21	2011	8	737094

Query executed successfully.

--Below is the 3-month rolling average calculations done using a sub-query

-- I will put this into a CTAS to use in Tableau for insights

```
select
*,
avg(totalrevenueforMonth) over(order by yyear, mmonth rows between 2 preceding and current row)
ThreeMonthRollingAverage
from
(
   select
      datepart(year,invoicedate)  yyear,
      datepart(month,invoicedate) mmonth,
      sum(totalrevenue) totalrevenueforMonth
   from onret
   group by datepart(year,invoicedate), datepart(month,invoicedate)
)t;
```

Results | Messages

	yyear	mmonth	totalrevenueforMonth	ThreeMonthRollingAverage
1	2009	12	794499	794499
2	2010	1	633114	713806
3	2010	2	534780	654131
4	2010	3	807463	658452
5	2010	4	659404	667215
6	2010	5	634923	700596
7	2010	6	724453	672926
8	2010	7	629580	662985
9	2010	8	675107	676380
10	2010	9	895282	733323
11	2010	10	1125598	898662
12	2010	11	1410844	1143908
13	2010	12	790524	1108988
14	2011	1	669006	956791
15	2011	2	506578	655369
16	2011	3	695147	623577
17	2011	4	516726	572817
18	2011	5	748654	653509
19	2011	6	735137	666839
20	2011	7	696588	726793

Query executed successfully. DESKTOP-1MKC00T\SOLEX

```
select
rank() over(order by yyear, mmonth) timePoint,
*,
```

```
avg(totalrevenueforMonth) over(order by yyear, mmonth rows between 2 preceding and current row)
ThreeMonthRollingAverage

into ThreeMonthRollingAverage

from

(

   select

      datepart(year,invoicedate)  yyear,

      datepart(month,invoicedate) mmonth,

      sum(totalrevenue) totalrevenueforMonth

   from onret

   group by datepart(year,invoicedate), datepart(month,invoicedate)

)t;
```

	timePoint	yyear	mmonth	totalrevenuefo...	ThreeMonthR...
▶	1	2009	12	794499	794499
	2	2010	1	633114	713806
	3	2010	2	534780	654131
	4	2010	3	807463	658452
	5	2010	4	659404	667215
	6	2010	5	634923	700596
	7	2010	6	724453	672926
	8	2010	7	629580	662985
	9	2010	8	675107	676380
	10	2010	9	895282	733323
	11	2010	10	1125598	898662
	12	2010	11	1410844	1143908
	13	2010	12	790524	1108988
	14	2011	1	669006	956791
	15	2011	2	506578	655369
	16	2011	3	695147	623577
	17	2011	4	516726	572817
	18	2011	5	748654	653509
	19	2011	6	735137	666839
	20	2011	7	696588	726793
	21	2011	8	737094	722939
	22	2011	9	1027866	820516
	23	2011	10	1109474	958144
	24	2011	11	1446705	1194681
	25	2011	12	620691	1058956
*	NULL	NULL	NULL	NULL	NULL

```
-- I have added a timePoint and created a new physical table to use in tableau.
-- The timepoint should make it easier for Tableau to create a time series

select * from ThreeMonthRollingAverage;
drop table ThreeMonthRollingAverage;




select
*,
avg(sum(cast(totalrevenue as float))) over(partition by datepart(year,invoicedate),
datepart(month,invoicedate) order by datepart(year,invoicedate), datepart(month,invoicedate) rows
between 2 preceding and current row) ThreeMonthRollingAverage
from onret
```

```
order by datepart(year,invoicedate), datepart(month,invoicedate)
;

select * from onret;
```

-- Next I will calculate what percentage of revenue that comes from the top 20% of customers by revenue. I will use ntile(5), a view and a subquery

```
create view TRCustomerRevenue as
(
   select
      ntile(5) over(order by totrev desc) trank,
      *
```

```
    from
    (
        select
            customer_ID,
            sum(totalrevenue) totrev
        from onret
        where customer_ID != 'guest'
        group by Customer_ID
    )t
)
```

Results | Messages

	trank	customer_ID	totrev
1	1	18102	580478
2	1	14646	526679
3	1	14156	311410
4	1	14911	286492
5	1	17450	244581
6	1	13694	194982
7	1	17511	171254
8	1	16446	168471
9	1	16684	146806
10	1	12415	144082
11	1	15061	125975
12	1	16029	117343
13	1	17949	116084
14	1	15311	112502
15	1	13089	111886

Query executed successfully.

```
--I'll use these to check my query
select sum(totrev) from TRCustomerRevenue where trank =1;
select sum(totrev) from TRCustomerRevenue;
```

Results | Messages

	(No column name)
1	13112682

	(No column name)
1	16870376

--Query below calculates the answer (as 0.78)

select top 1 cast((select sum(totrev) from TRCustomerRevenue where trank =1) as float)/cast((select sum(totrev) from TRCustomerRevenue)as float) from TRCustomerRevenue;

Results | Messages

	(No column name)
1	0.777260803197273

--Next I will create a customer lifetime revenue distribution using case on the same view created above

```
select
*,
case
   when totrev between 0 and 999 then '$0-$1000'
   when totrev between 1000 and 2999 then '$1000-$3000'
   when totrev between 3000 and 4999 then '$3000-$5000'
   when totrev between 5000 and 6999 then '$5000-$7000'
   when totrev between 7000 and 8999 then '$7000-$9000'
   when totrev between 9000 and 9999 then '$9000-$10000'
   else '$10000+'
END revenueBand
from TRCustomerRevenue;
```

Results | Messages

	trank	customer_ID	totrev	revenueBand
286	1	15981	9233	$9000-$10000
287	1	13162	9182	$9000-$10000
288	1	15719	9158	$9000-$10000
289	1	14796	9146	$9000-$10000
290	1	14292	9140	$9000-$10000
291	1	14085	9059	$9000-$10000
292	1	13735	9025	$9000-$10000
293	1	15249	9013	$9000-$10000
294	1	18109	9011	$9000-$10000
295	1	13327	8994	$7000-$9000
296	1	15547	8985	$7000-$9000
297	1	17315	8916	$7000-$9000
298	1	15903	8909	$7000-$9000
299	1	14367	8895	$7000-$9000
300	1	15298	8886	$7000-$9000
301	1	15727	8815	$7000-$9000
302	1	14060	8799	$7000-$9000
303	1	16709	8783	$7000-$9000
304	1	14135	8758	$7000-$9000
305	1	16532	8755	$7000-$9000

-- I will now do a 90 day churn detection analysis relative to the last purchase date in the dataset

-- The final table will return all customers, the gap between their last purchase and the latest date on the dataset and a flag if they haven't purchase for over 90 days

```
select
*,
case
   when orderGap>90 then 'churn detected'
   else 'ok'
   end churn
from
(
   select
      customer_ID,
      DATEDIFF(day, max(invoicedate), (select max(invoicedate) mx from onret)) orderGap
```

```
    from onret
    group by Customer_ID
)t
order by ordergap desc;
```

Results | Messages

	customer_ID	orderGap	churn
2...	16723	91	churn detected
2...	14414	91	churn detected
2...	12535	91	churn detected
2...	16027	91	churn detected
2...	13441	91	churn detected
2...	14628	91	churn detected
2...	14915	91	churn detected
2...	15652	91	churn detected
2...	12965	89	ok
2...	12635	89	ok
2...	18215	89	ok
2...	14212	89	ok
2...	13124	89	ok
2...	16796	89	ok
2...	18255	89	ok
2...	15035	89	ok
2...	15939	89	ok
2...	13372	89	ok
3...	14837	89	ok
3...	13611	89	ok

Query executed successfully.

-- Here is another version of the above but only retruning customers with churn detected

```
select * from
(
   select
      *,
      case
         when orderGap>90 then 'churn detected'
         else 'ok'
         end churn
   from
(
select
   customer_ID,
   DATEDIFF(day, max(invoicedate), (select max(invoicedate) mx from onret)) orderGap
from onret
group by Customer_ID
```

```
)t
)s
where ordergap>90
order by ordergap desc;
```

Results | Messages

	customer_ID	orderGap	churn
1	17056	738	churn detected
2	14654	738	churn detected
3	12636	738	churn detected
4	17592	738	churn detected
5	13526	738	churn detected
6	15833	737	churn detected
7	14106	737	churn detected
8	14980	737	churn detected
9	17606	737	churn detected
10	17087	737	churn detected
11	17818	737	churn detected
12	17909	737	churn detected
13	17271	736	churn detected

-- Below I will calculate the average number of days between orders per customer. I have also included the total number of orders per customer

```
-- To give more meaning to those averages.

select
   customer_ID,
   avg(daysSinceLastOrder) avgDaysBetweenOrders,
   count(distinct invoice) noOfOrders
from
(
   select
      *,
      datediff(day,(lag(invoicedate) over(partition by customer_ID order by invoicedate)),invoicedate)
daysSinceLastOrder
   from onret
)t
group by customer_ID
order by avgDaysBetweenOrders desc;
```

Results | Messages

	customer_ID	avgDaysBetweenOrders	noOfOrders
1	18174	596	2
2	16737	524	2
3	17715	392	2
4	13068	309	2
5	18080	224	2
6	16990	204	2
7	18273	196	4
8	15603	190	2
9	16454	189	4
10	13302	185	2
11	15461	171	2
12	17310	147	2
13	14865	146	4

-- Below is an analysis by country.

-- Avereage revenue per countr

-- standard deviation of revenue per country

-- co-efficient of variation within each country

-- I have used a CTE to first seperate revenue grouped by countries

```
with CountryAnalysis as
(
   select
      country,
      invoice,
      sum(quantity*price) tr
   from onret
   group by country, invoice
)
select
   country,
   count(distinct invoice) noOfOrders,
   avg(tr) avgRevCountry,
   stdev(tr) stDevCountry,
   stdev(tr)/avg(tr) CofV
from CountryAnalysis
```

group by country;

	country	noOfOrders	avgRevCountry	stDevCountry	CofV
1	Finland	57	521.472630650851	429.729840691533	0.82406978896512
2	Bahrain	10	310.978998196125	166.693272892973	0.536027429054371
3	EIRE	626	1048.67237503014	2349.25343022506	2.24021675993663
4	USA	20	413.262998962402	371.766601489716	0.899588403566559
5	Italy	65	493.971844273347	394.238710041538	0.798099556911951
6	Brazil	2	704.63499648124	620.79025248197	0.881009679595864
7	Bermuda	1	1253.13997983932	NULL	NULL
8	Netherlands	228	2428.87201132377	3946.61945991381	1.62487749107984
9	Hong Kong	15	1558.35467786789	933.767513891022	0.599200892551999
10	Germany	789	537.051443763859	596.384893578711	1.11048001174528
11	Lithuania	6	815.44666069746	742.363285534959	0.910376265321889
12	Iceland	8	615.191248975694	315.773657055726	0.513293480005601
13	RSA	4	1162.92001235485	236.575504871275	0.203432310354882

```
select
    country,
    stdev(totalrevenue) stDevC,
    avg(totalrevenue) avgRevenue,
    stdev(totalrevenue)/avg(totalrevenue) CofV
```

```
from onret
group by country

-- Finally I will create an analysis of the running total of revenue using a sum() window function within a frame and also calculate the
-- percentage of total sales to the same point by adding a subqueried column

select
    *,
    sum(totalrevenue) over(order by invoicedate rows between unbounded preceding and current row) cumulativeRevenue,
    (sum(totalrevenue) over(order by invoicedate rows between unbounded preceding and current row)*100)/cast((select sum(totalrevenue) from onret)as float) percentageOfTotalRev
from onret
order by invoicedate;
```

Results | Messages

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer_ID	Country	totalRevenue	cumulativeRevenue	percentageOfTotalRev
1	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00.000	1.25	13085	United Kingdom	30	30	0.000151322276752606
2	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01 07:45:00.000	5.94999980926514	13085	United Kingdom	59	89	0.000448922754366064
3	489434	21871	SAVE THE PLANET MUG	24	2009-12-01 07:45:00.000	1.25	13085	United Kingdom	30	119	0.00060024503111867
4	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00.000	2.09999990463257	13085	United Kingdom	100	219	0.00110465262029402
5	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01 07:45:00.000	1.64999997615814	13085	United Kingdom	39	258	0.00130137158007241
6	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00.000	6.75	13085	United Kingdom	81	339	0.00170994172730445
7	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00.000	6.75	13085	United Kingdom	81	420	0.00211851187453648
8	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00.000	6.94999980926514	13085	United Kingdom	83	503	0.00253717017355202
9	489435	22195	HEART MEASURING SPOONS LARGE	24	2009-12-01 07:46:00.000	1.64999997615814	13085	United Kingdom	39	542	0.00273388913333041
10	489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-12-01 07:46:00.000	3.75	13085	United Kingdom	45	587	0.00296087254845932
11	489435	22350	CAT BOWL	12	2009-12-01 07:46:00.000	2.54999995231628	13085	United Kingdom	30	617	0.00311219482521193
12	489435	22353	LUNCHBOX WITH CUTLERY FAIRY CAKES	12	2009-12-01 07:46:00.000	2.54999995231628	13085	United Kingdom	30	647	0.00326351710196453
13	489436	21181	PLEASE ONE PERSON METAL SIGN	12	2009-12-01 09:06:00.000	2.09999990463257	13078	United Kingdom	25	672	0.00338961899925837

Executing query